

SP 27 – Red Spotify

Project Development Document

CS 4850/Section 02, Spring Semester 2026

Professor Perry

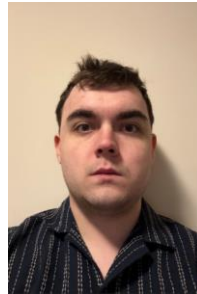
Team Members:



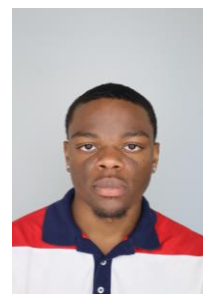
Davon
Appolon



Rowan
Crooks-
Taylor



Andrew
Henshaw



Isaiah
Langford

Table of Contents

1. Implementation Detail Narrative.....	3
1.0 System Integration & Workflow Overview	3
1.1 Component Name	3
1.2 Component Name	3
2. Database and External Connections	3
2.1 Connection Overview	3
2.2 Configuration Details.....	3
2.3 Integration Logic.....	4
3. Project Setup and Environment Configuration	5
3.1 Prerequisites.....	5
3.2 Installation Steps	5
3.3 Verification	5
4. Development Conclusion or Summary	5

1. Implementation Detail Narrative

1.0 System Integration & Workflow Overview

The Integrated Process:

The development of our Spotify app is made up of three main parts:

1. **Frontend:** Handles the user interface and sends requests.
2. **Backend service:** Securely handles authentication and token exchange.
3. **Spotify API:** Provides user data such as profile info, playlists, and listening activity.

Operational Workflow

1. User opens app
2. User selects login
3. User selects "Login with Spotify" button
4. App redirects the user to Spotify login page
5. Spotify returns authentication code
6. Frontend sends code to the backend
7. Backend exchanges code for access and refresh tokens
8. Frontend uses the access token to request data from Spotify
9. Spotify returns user data
10. Data displayed in app
11. If token expires, it is refreshed automatically

Global Design Decisions

- OAuth 2.0 used for secure authentication.
- Backend used to protect sensitive data.
- Error handling used to manage missing/failed API data.
- React Native used for cross platform development.
- Reusable API request functions to simplify development.

1.1 Authentication

Conceptual Overview: Authentication handles user login and authorization with Spotify OAuth. This lets the user connect their Spotify account to the app.

Development Process:

- Set up Spotify authorization and token endpoints.
- Implement login with Expo AuthSession.

1.2 API Integration

Conceptual Overview: API integration deals with retrieving data from the Spotify Web API. This includes the user's profile, playlists, and music data.

Development Process:

- Create functions to send API requests.
- Add authorization headers with access token.
- Retrieve and parse data.
- Build reusable functions for different endpoints.
- Error handling for failed requests.

1.3 Frontend UI

Conceptual Overview: The Frontend UI displays all data to the user such as the dashboard, search, and library features.

Development Process:

- Develop screens using React Native components
- Implement navigation between pages
- Connect UI with API data
- Add crash / error messages

1.4 Backend

Conceptual Overview:

The backend of the application handles secure communication with the Spotify API for authentication and token exchange.

Development Process:

- Backend built on Node
- Connect backend to Spotify API
- Store user credentials securely using environment variables
- Test authentication flow

2. Database and External Connections

2.1 Connection Overview:

This application uses the external connection, Spotify Web API. Which provides data specific to the user, which includes profile information, playlists, and listening activity.

Additionally, the system utilizes a custom backend service hosted on Render that securely handles authentication. The backend acts as an intermediary between the React frontend and the Spotify authorization servers.

Client-Server Model Architecture:

- Frontend (Expo React Native app) handles UI and user interaction
- Backend handles secure token exchange
- Spotify API provides user data

2.2 Configuration Details:

The application requires these configuration elements to establish the external connections:

Spotify API Endpoints

- Authorization Endpoint
 - <https://accounts.spotify.com/authorize>
- Token Endpoint
 - <https://accounts.spotify.com/api/token>
- API Base URL
 - <https://api.spotify.com/v1/>

Backend Endpoint

- Hosted backend callback URL
 - <https://sp-27-spotifyapp-red.onrender.com/callback>
- Token exchange endpoint
 - /auth/token

Environment Variables

The following environment variables are required:

SPOTIFY_CLIENT_ID=your_client_id

This variable is used to authenticate the application with Spotify without leaving sensitive credential information in the codebase.

Authentication Configuration

- OAuth 2.0 Authorization Code Flow with PKCE is used
- Redirect URI is dynamically generated using Expo AuthSession
- Scopes requested include:
 - user profile access
 - email access
 - playback and listening data

2.3 Integration Logic:

Spotify is integrated with this application using a multi-step authentication and data retrieval process

1. User Authentication

- The user initiates login through the frontend
- The app redirects the user to the Spotify authorization page
- If the login is successful, Spotify returns an authorization code

2. Token Exchange (Backend)

- The frontend sends the authorization code to the backend
- Backend exchanges the code for:
 - Access token
 - Refresh token
- Tokens are returned to the frontend and stored in application state

3. Authenticated API Requests

- Frontend uses access tokens to make requests to Spotify endpoints
- Requests include:
 - Authorization: Bearer <access_token>
- Data retrieved includes user profile and music related information

4. Token Refresh

- When access token expires, the refresh token is used to gain a new one
- Ensures continuous access without requiring the user to login again

3. Project Setup and Environment Configuration

3.1 Prerequisites:

1. Software requirements
 - Node.js version 18+
 - Expo Client (installed in project)
 - Spotify Web API
 - Visual Studio Code
 - Expo Go Mobile App (for testing)
2. Hardware Requirements:
 - Android 10+ or iOS 14+
 - A PC to run the Expo app server
3. Accounts/API Access
 - Valid Spotify account (Spotify premium)
 - Spotify developer account (for API)

3.2 Installation Steps: Provide a numbered list of commands or steps to get the project running.

1. Clone repository
 - a. Install Git
 - a. Create a local directory folder for the project, and cd to it
 - b. Enter “git clone <https://github.com/rowancrookstaylor/SP-27-SpotifyApp-Red.git>”
2. Install Project Dependencies
 - a. Install Node.js. Verify in command window with “node -v” and “npm -v”
 - b. CD to the repo’s main folder, and Install Expo Client with “npx expo install expo-dev-client”
 - c. Install other dependencies with “npm install”
3. Download ExpoGo for your mobile device
4. Run the app in the folder directory with “npx expo start”
5. Scan QR code with your mobile device to add the Expo app to your ExpoGo app.

3.3 Verification: How can the user confirm if the project is set up correctly? Describe the expected output of a successful launch.

1. Application Launch

- When the application launches, the user should see the login button and the navigation tabs at the bottom of the screen. The app may throw an invalid token error, but that is only because the refresh token may not be set yet.

2. Login

When the user selects the login button, the application will direct them to the Spotify authorization page to link their accounts. After this, the user should be redirected back to the application without any errors.

3. User Data retrieved

When the user logs in, their data from Spotify will be retrieved and displayed on the application. This will include their profile, playlists, and listening activity.

4. Development Conclusion or Summary

The existing Red Spotify app gets the core system up and running. The React Native frontend, backend authentication service, and Spotify Web API all work together. The app logs in users with the full Spotify OAuth 2.0 flow. It fetches user data from the Spotify API and displays it on the client-side.

The system consists of a client (the app's frontend), a server (the app's authentication service), and an API (the Spotify Web API). The app can be run locally with Expo and Node.js. The backend service is deployed to Render, where it can communicate securely with Spotify's servers.